



ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

COURSE CODE : EEE 4308

COURSE NAME : DIGITAL ELECTRONICS LAB

PROJECT: SAP-1 Architecture based 8-bit Computer Design and Implementation.

Phase: A

GROUP NAME: Boolean Squad

GROUP MEMBERS:

- IKRAM HOSSAIN (230021111)
- NADMAN IBNE MAMUN (230021114)
- FARHAN ISHRAQ (230021124)
- SAMIN YASAR SAMI (230021138)
- ZAVID ATAHAR SAMIN (230021155)
- AGHA MAKINUN AMIN CHOWDHURY (230021232)
- RAZEEN ABRAR RAHMAN (230021215)

Supervisor Name:

FATEMA TASNIM OYSHI

Junior Lecturer,
Department of EEE,
IUT

OBJECTIVES:

The objective of phase A is to design and implement the following parts:

1. Program Counter
2. Clock Pulse generator

DESCRIPTION:

CLOCK PULSE GENERATOR: The clock pulse generator is the timing center of the computer. It produces a continuous stream of square waves that synchronize all operations within the system.

Operation Modes:

1. MANUAL: It allows for single-stepping through instructions using a push button.
2. AUTO: Generates a steady stream frequency to execute programs at a set speed.

Function: It provides CLK signals to the registers, the program counter and the control-sequencer to ensure data is moving at correct intervals.

PROGRAM COUNTER: The program counter is a 4-bit counter that tracks the memory address of the next instruction to be fetched. It counts from 0000_2 to 1111_2 (0 to 15 in decimal).

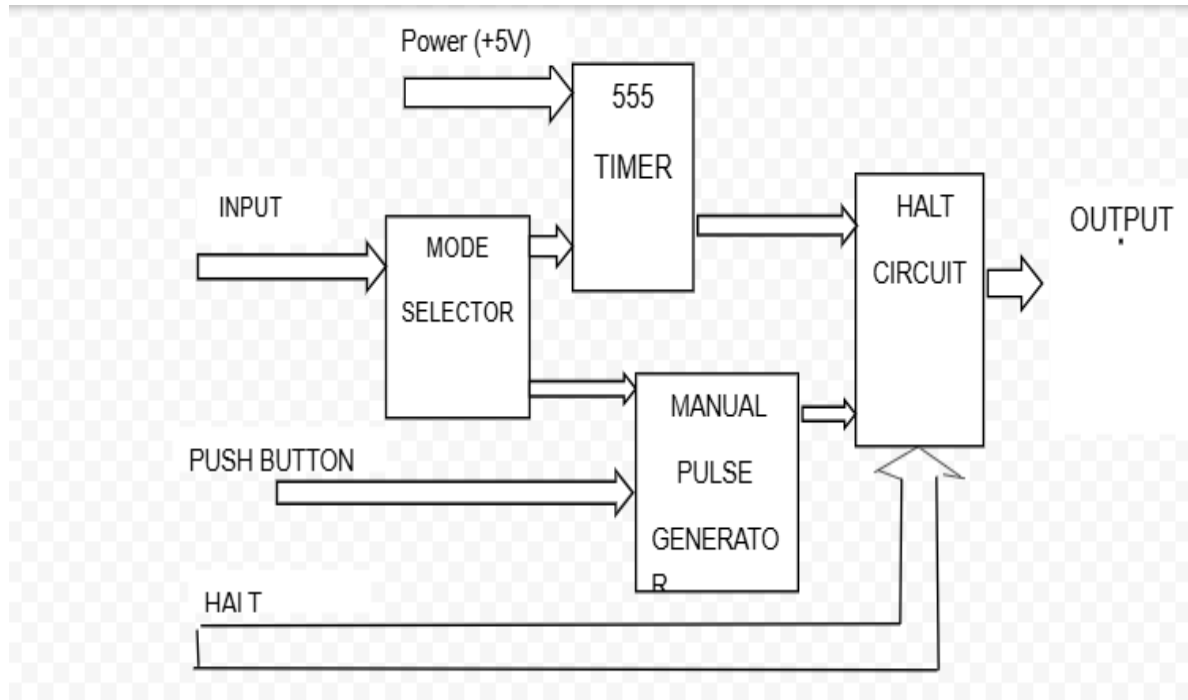
Control Signals:

1. CE: Increments the program counter on the next clock signal.
2. CO: It puts the current counter value on the bus.
3. RESET: It resets the counter to 0000.
4. CLK: It is the clock pulse input.

Function: It ensures that the computer executes instructions in the correct sequence.

CLOCK PULSE GENERATOR

BLOCK DIAGRAM:



TRUTH TABLE:

MODE (S)	Halt(H)	Auto Pulse (A)	Manual Pulse(M)	Output(Y)
X	1	X	X	0
1(auto)	0	0	X	0
1(auto)	0	1	X	1
0(manual)	0	X	0	0
0(manual)	0	X	1	1

BOOLEAN EXPRESSION:

The Boolean expression derived from this truth table is given below:

$$Y = \overline{H} \cdot ((\overline{S} \cdot M) + (S \cdot A))$$

Development of Clock Pulse Generator

Introduction to 555 Timer IC:

- Pin 1. – **Ground**
- Pin 2. – **Trigger**
- Pin 3. – **Output**
- Pin 4. – **Reset**
- Pin 5. – **Control Voltage**
- Pin 6. – **Threshold**
- Pin 7. – **Discharge**
- Pin 8. – **Supply**

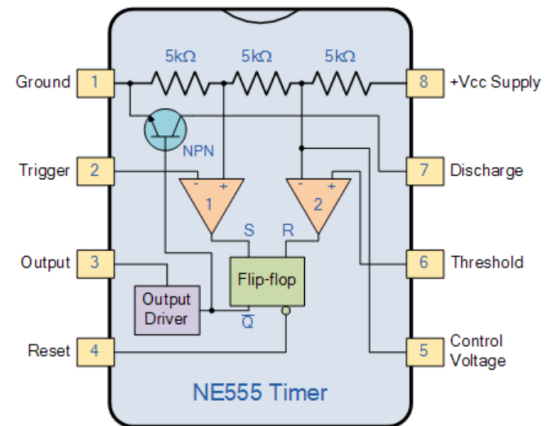
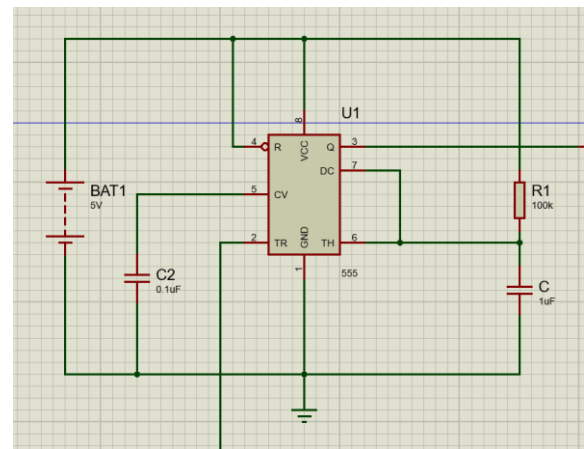


Fig. Pin diagram of 555 Timer IC

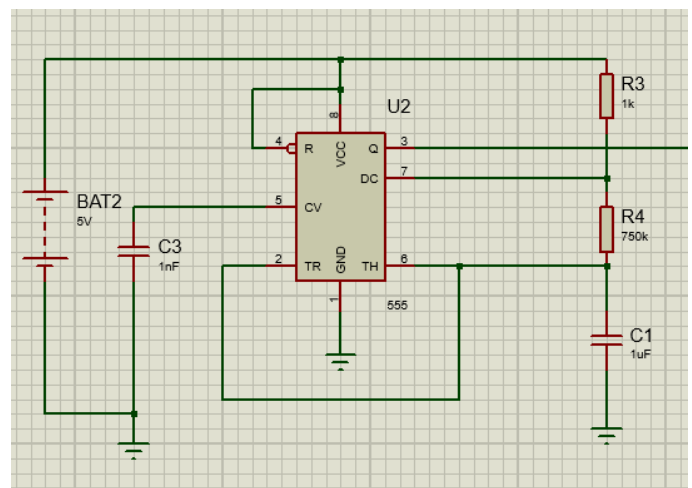
Monostable Multivibrator using 555 Timer IC:

The circuit given in the right side is responsible for the manual mode of our clock pulse generator. Here the Pin2 or the trigger pin is connected to a triggering circuit which is described later.



Monostable Multivibrator using 555 Timer IC:

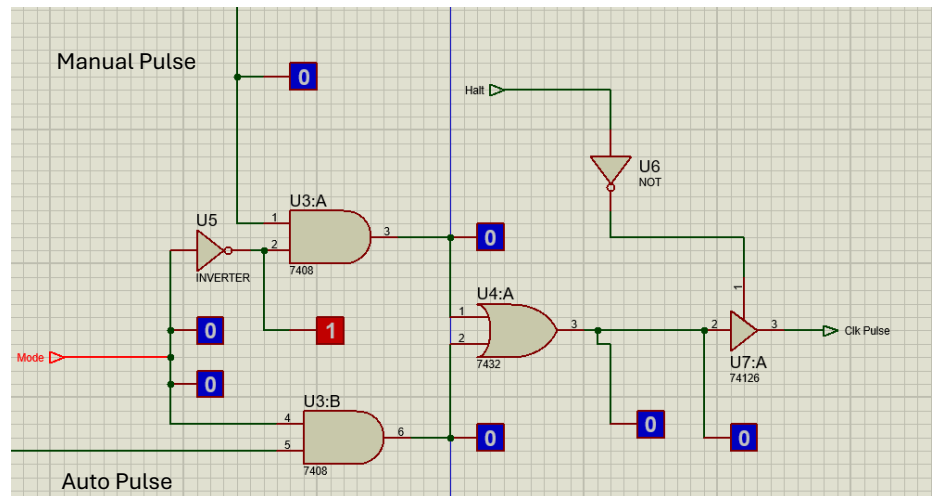
The circuit given in the right side is responsible for the auto mode of our clock pulse generator.



Mode Selection Circuit:

Mode=0 Manual pulse gets through

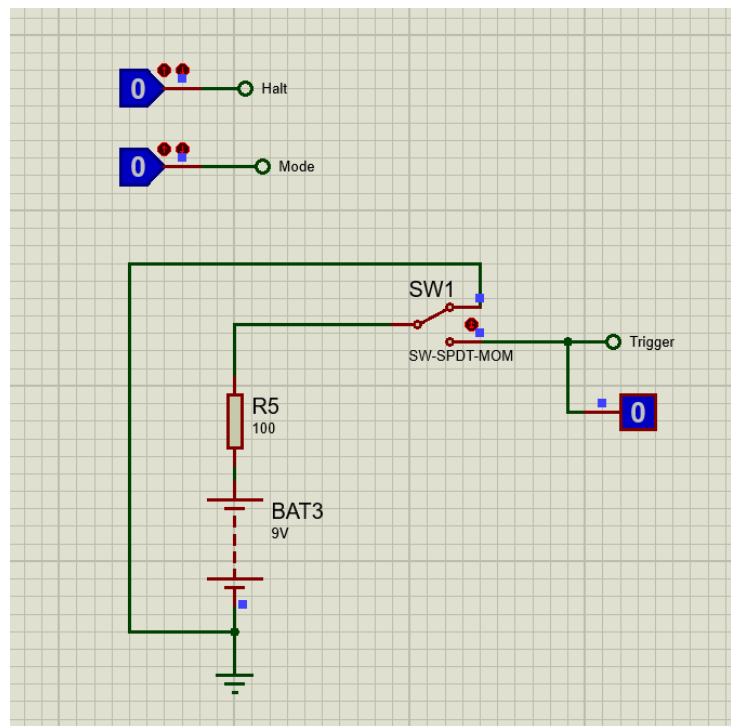
Mode=1 Auto pulse gets through



Triggering Circuit and Control Center:

Our triggering method consists of a simple circuit with a SPDT momentary switch that changes its position for a moment which results in a flow of current in the pin2 of our manual pulse generator or mono-stable multivibrator circuit.

Again there is Halt to stop the clock pulse and Mode button to control the mode of operation. It should be mentioned that Halt=1 indicates the pulse stops, Mode=0 indicates Manual pulse and Mode=1 indicates Auto pulse.



Overall Circuit Design:

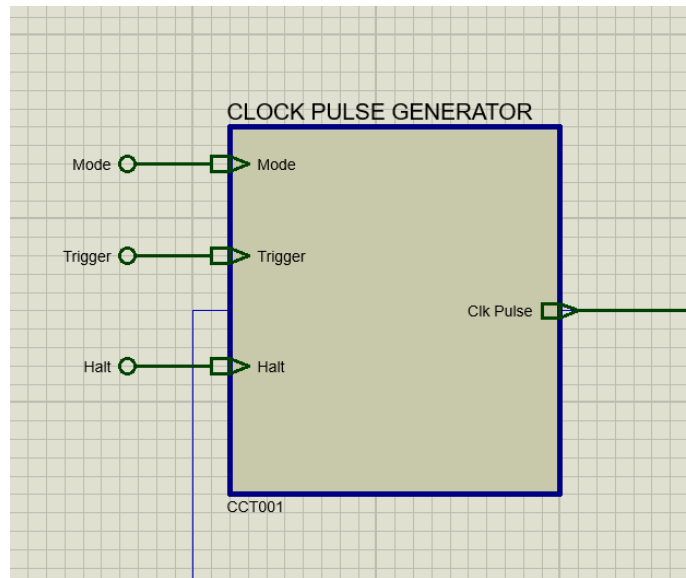


Fig. Subcircuit diagram of Clock Pulse Generator

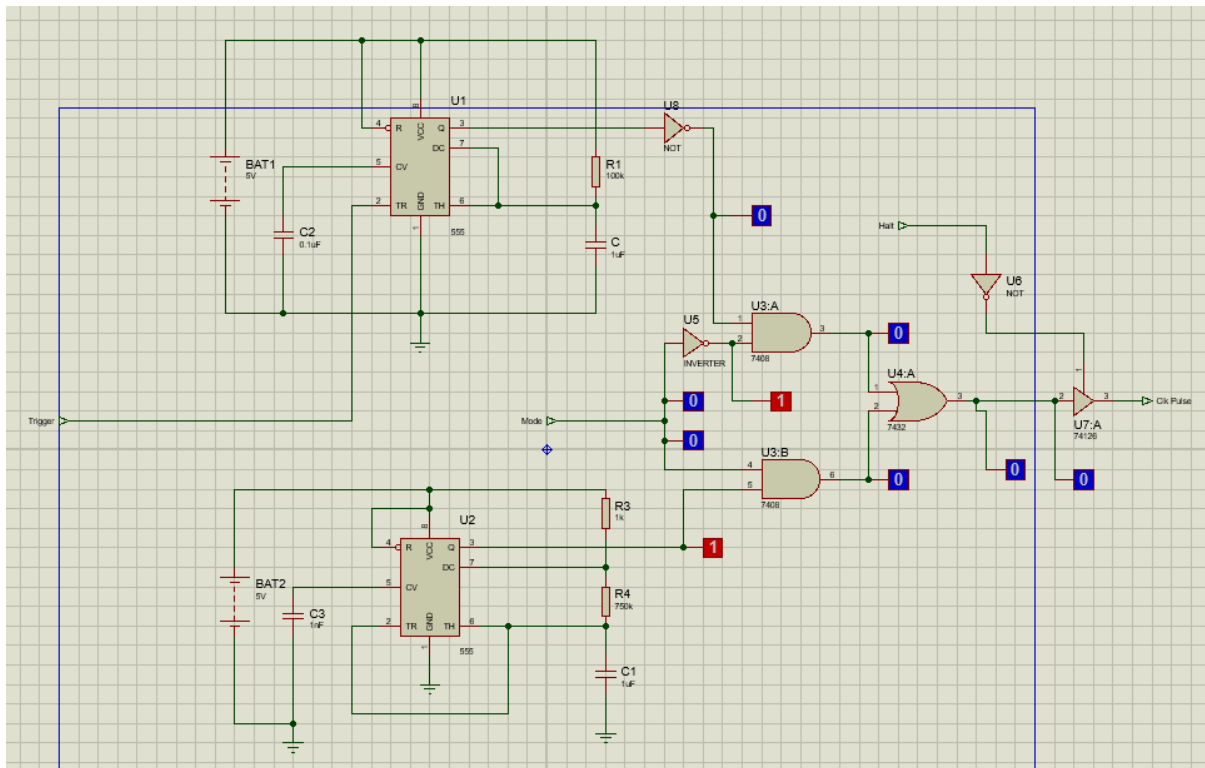


Fig. childsheet diagram of Clock Pulse Generator

Equipment / Component List

1. OR Gate IC(74HC32), AND Gate IC (74HC08), NOT Gate IC (74HC04)

Used in the selection circuit.

2. Tri-State Buffer (74126)

Used to interface the programme counter with the system bus and prevent bus contention.

3. 555 Timer IC (Used as the main pulse generator)

4. Resistors

5. Capacitor (non electrolytic)

6. Battery (5V DC)

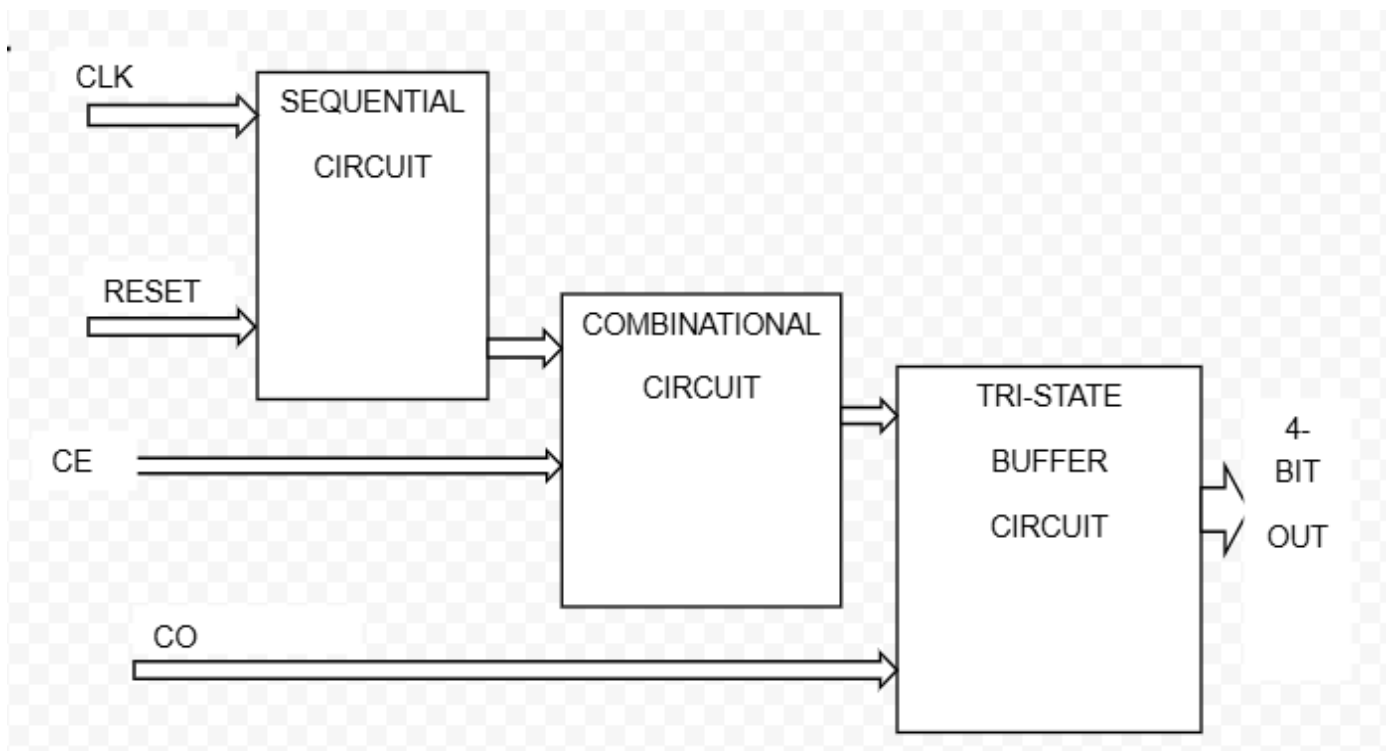
Required to power ICs used in the circuit.

7.Connecting Wires

Used to interconnect all components in the circuit.

PROGRAM COUNTER

In this project, an 8-bit synchronous program counter has been designed for the SAP-1 computer to generate sequential instruction addresses in a reliable and controlled manner. Unlike an asynchronous counter, all JK flip-flops in this design receive the same clock signal, which ensures that every bit of the program counter updates at the same instant and eliminates ripple delay and temporary invalid states on the address bus. The JK flip-flops are operated in toggle mode, and AND gates are used to decide when each higher-order bit should toggle by checking whether all lower-order bits are high, allowing proper binary counting while remaining fully synchronous. A reset circuit is included to initialize the program counter to zero at the start of operation, ensuring correct program execution from the first instruction. The outputs of the program counter are connected to a tri-state buffer (74LS245), which allows the counter to place its value onto the system bus only when enabled, preventing bus contention with other components. Overall, this synchronous program counter provides stable, glitch-free address generation and integrates effectively with the SAP-1 control and bus structure.



TRUTH TABLE:

RESET	CE	CO	CLK	OUTPUT Q ₃ Q ₂ Q ₁ Q ₀	MODE
1	X	X	X	0 0 0 0	Reset
0	0	1	↑	No change	Hold
0	1	1	↑	Count +1	Increment
0	X	0	X	0 0 0 0	disconnect

BOOLEAN EXPRESSION:

The output expressions for the 4 bits are given below:

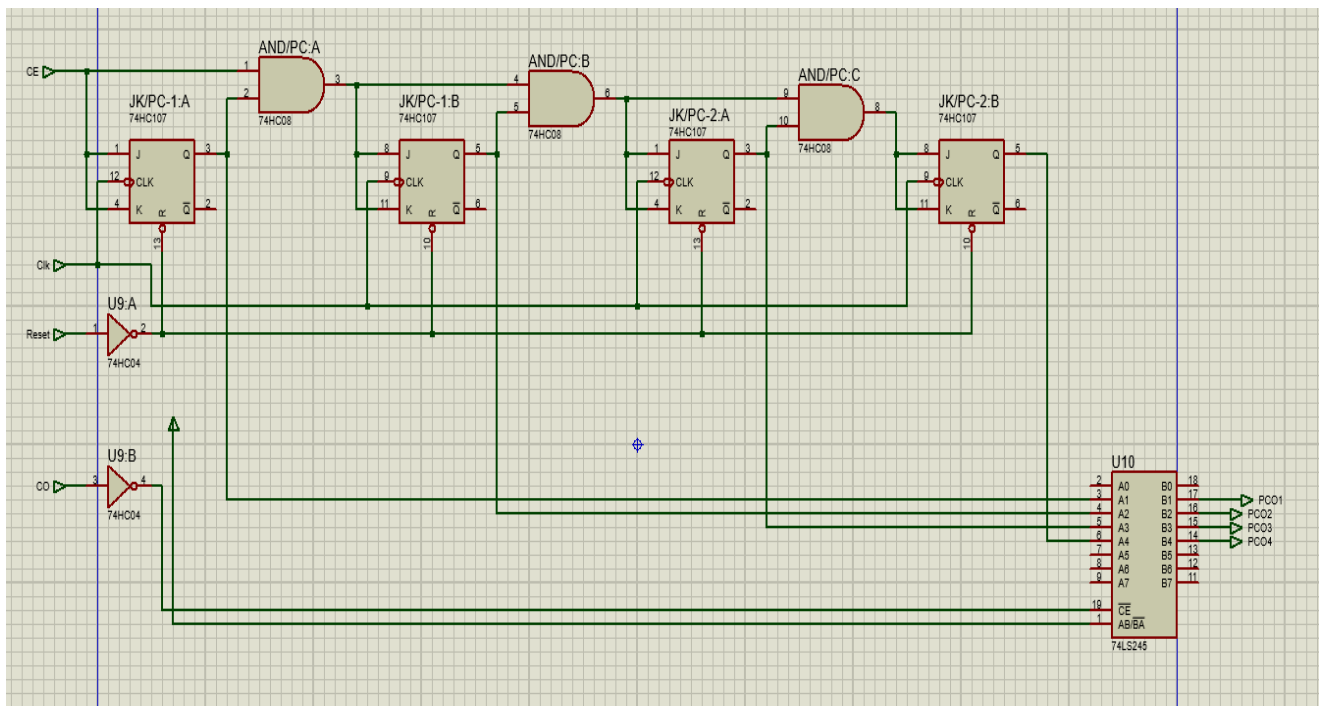
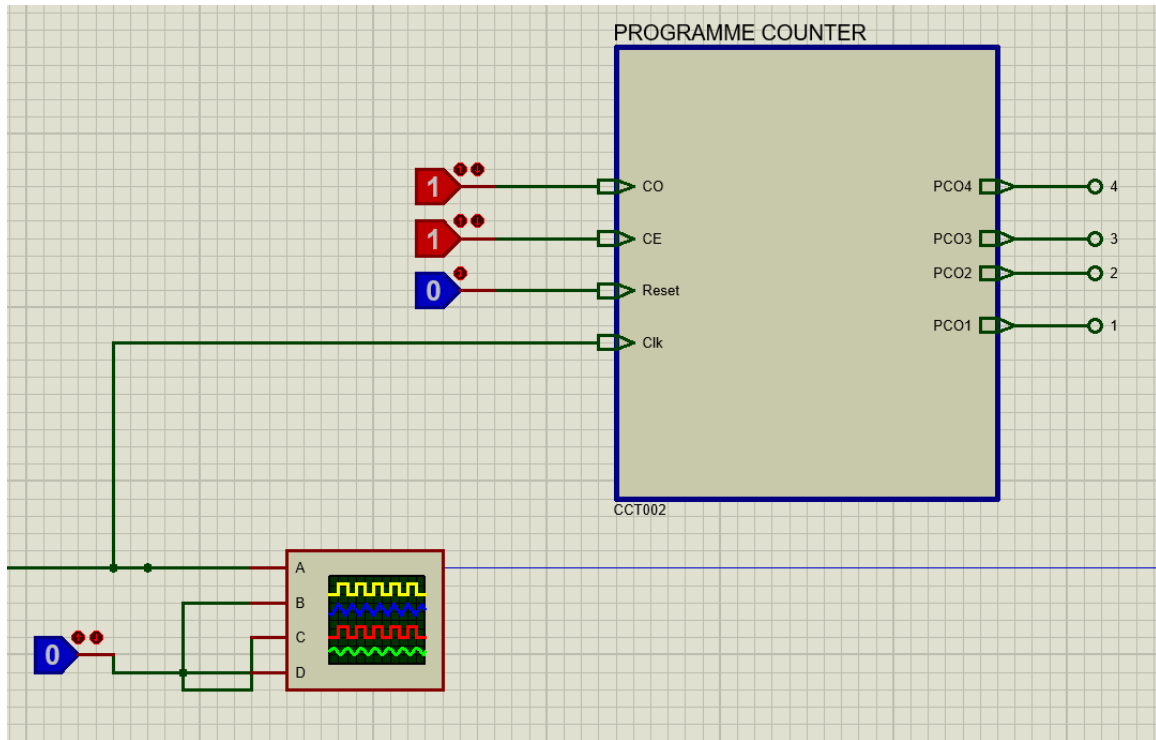
$$Q_0^{next} = \overline{R} \cdot (CE \oplus Q_0)$$

$$Q_1^{next} = \overline{R} \cdot ((CE \cdot Q_0) \oplus Q_1)$$

$$Q_2^{next} = \overline{R} \cdot ((CE \cdot Q_0 \cdot Q_1) \oplus Q_2)$$

$$Q_3^{next} = \overline{R} \cdot ((CE \cdot Q_0 \cdot Q_1 \cdot Q_2) \oplus Q_3)$$

Circuit Diagram:

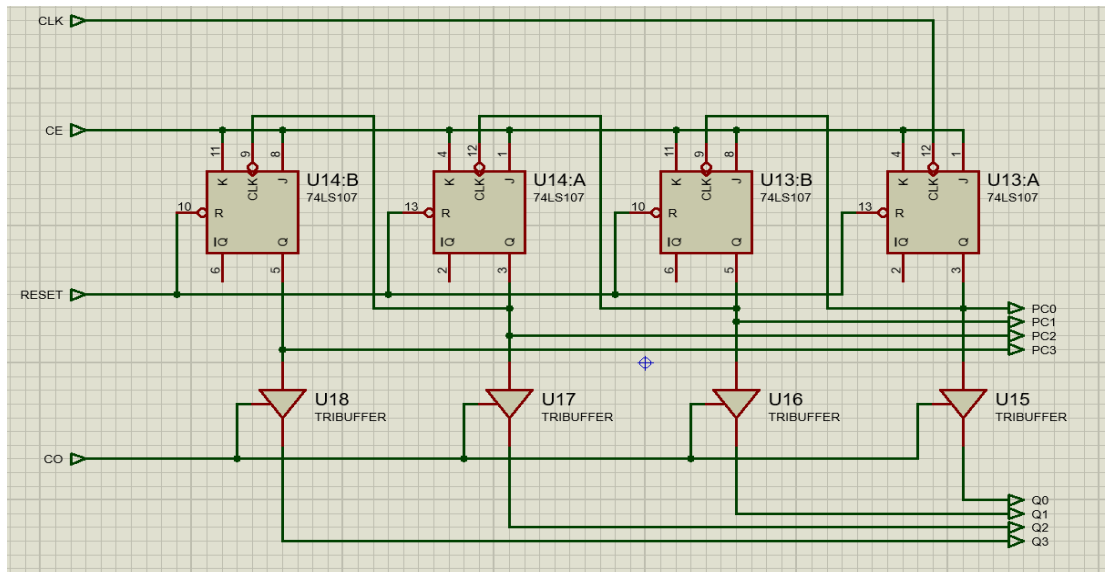


Equipment / Component List

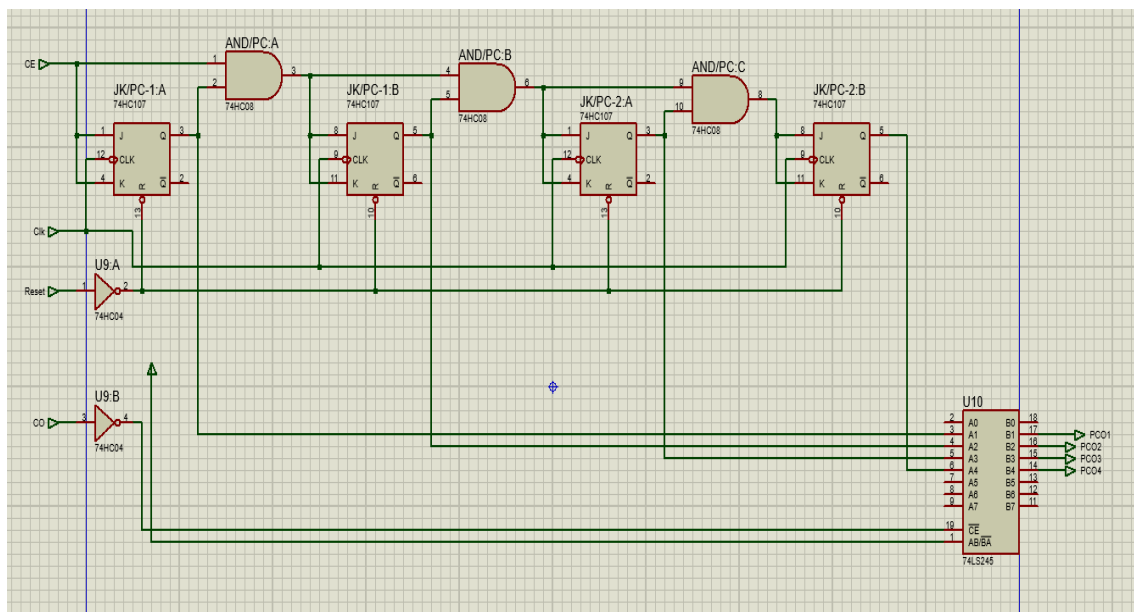
1. **JK Flip-Flop IC (74HC107)**
Used to store and toggle the binary count of the programme counter in synchronous mode.
2. **AND Gate IC (74HC08)**
Used to generate the required toggle conditions for higher-order JK flip-flops in synchronous counting.
3. **NOT Gate IC (74HC04)**
Used for signal inversion, mainly for reset and control signal conditioning.
4. **Tri-State Buffer IC (74LS245)**
Used to interface the programme counter with the system bus and prevent bus contention.
5. **Clock Pulse Generator**
Provides the common clock signal required for synchronous operation of all flip-flops.
6. **Reset Switch / Reset Signal Source**
Used to initialize the programme counter to zero.
7. **Control Signals (CE, CO, AB/BA)**
Used to enable counting and control data flow from the programme counter to the bus.
8. **Power Supply (5V DC)**
Required to power all TTL and CMOS logic ICs used in the circuit.
9. **Connecting Wires**
Used to interconnect all components in the circuit.

Problems Faced:

● Program Counter



Initial

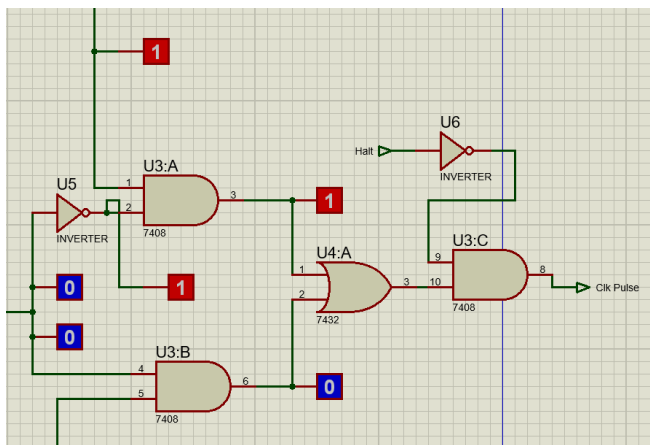


Final

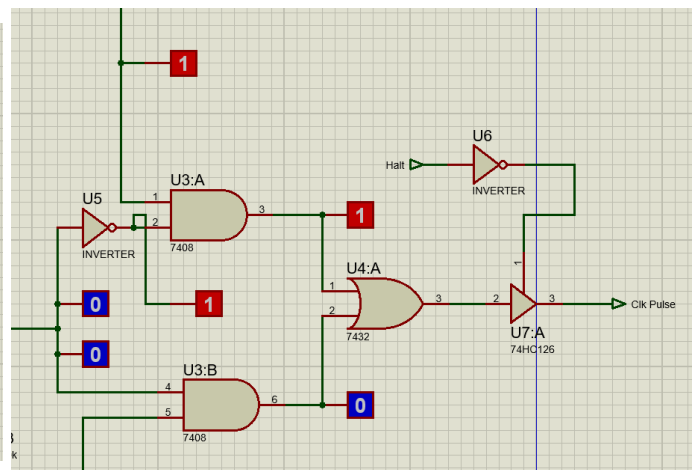
At the initial stage of the programme counter design, an asynchronous counter was used along with four individual tri-state buffers to place the counter output onto the bus. This approach created timing instability because the counter outputs did not change simultaneously, which could lead to temporary incorrect address values on the bus. Additionally, proper signal conditioning was not applied at first, as NOT gates were not used for the reset and enable control

signals. Due to this, the programme counter did not reset reliably and sometimes responded incorrectly to control inputs. These issues highlighted the limitations of the asynchronous design for a bus-based system like SAP-1. To solve these problems, the design was modified by implementing a synchronous counter with proper AND-gate controlled toggle logic and correct use of NOT gates for reset and enable signals, resulting in stable, predictable, and glitch-free operation.

● Clock Pulse Generator



Initial



Final

Due to our Mode selection circuit, when And Gate IC was used to Halt the Pulse there were some unwanted pulse seen in the output which could result in unwanted output for example the Program Counter connected to it could increment its value. To solve the problem, we just changed the And Gate IC with Tri state buffer (74HC1256).

Discussion:

Outcomes

The overall final design of the Clock Pulse Generator using 555 Timer IC can be considered as a reliable pulse generator for our SAP1 8Bit computer. On the other hand, final implementation of the programme counter using a synchronous design resulted in stable and reliable address generation for the SAP-1 computer.

Compared to the earlier asynchronous approach, the synchronous counter eliminated ripple delay and prevented temporary invalid outputs on the address bus. All flip-flops updated simultaneously using a common clock, which ensured correct instruction sequencing. The use of standard logic ICs such as JK flip-flops, AND gates, and a tri-state buffer improved modularity and made the design easier to understand and debug. Overall, the synchronous programme counter successfully met the timing and reliability requirements of a bus-based SAP-1 architecture.

Limitations

Even though the Clock Pulse generator looks easy on a software simulation there are many factors to be considered in case of hardware implementation. And similarly, despite its improved performance, the designed programme counter has some limitations.

The use of discrete logic ICs increases the overall hardware complexity and wiring, which may become inefficient for larger bit-width counters. The usage of 555 timer can result in not being able to generate High or Precise Frequency. The counter also operates only in an increment mode and does not support advanced operations such as loading a jump address or decrementing. Additionally, the design relies on a fixed clock frequency and does not include power-saving or dynamic clock control features. These limitations make the current design suitable mainly for educational and basic computer architectures like SAP-1.

Future Development

Several improvements can be made to enhance the functionality of the Clock Pulse Generator and Programme Counter in future versions. In case of Pulse Generator, A parallel load feature can be added to support jump and branch instructions, making the system more flexible. The counter can also be redesigned using integrated synchronous counter ICs or programmable logic devices such as CPLDs or FPGAs to reduce hardware complexity. Adding clock gating or enable-based power control can improve efficiency. Furthermore, expanding the counter width and integrating it with a more advanced control unit would allow this design to be used in more complex computer architectures beyond SAP-1.

